

A WhatsApp assistant that reads like a person on the front desk

How to run a natural, free-form AI conversation on WhatsApp — no templates, no robotic notifications — carrying everything the customer typed into the form. Plus the separate, optional piece: nudging a lead who never replied.

- 01 **Two different things** — and only one needs templates
- 02 **Let the customer open the thread** — the template-free path
- 03 **Making it feel human** — chunking, timing, tone
- 04 **The bot itself** — wiring WhatsApp to the existing brain
- 05 **"Are you a bot?"** — the one question to plan for
- 06 **Cold outbound** — the only place templates appear
- 07 **Prerequisites** — the account setup, in order
- 08 **Schema changes** — concrete Prisma
- 09 **What it costs**
- 10 **Gotchas & build order**

Written against the existing AURUM codebase — Next.js 16, Prisma 7 on Postgres, DeepSeek. Nothing in this document has been implemented. Pricing and platform policy are Meta's and change; verify current figures before committing to a budget.

01 · Two different things, and only one needs templates

Everything confusing about WhatsApp comes from collapsing two separate things into one. They have almost nothing in common.

	The chatbot	Cold outbound
Who speaks first	The customer	You
What you may send	Anything. Free-form, natural, as many messages as you like	Only a pre-approved template
Approval needed	None	Meta reviews every template
Cost	Free	Paid per message
Feels like	A person on the front desk	A notification

The thing you actually want has no templates in it

A customer messages your number; for the next 24 hours you can reply with anything you like, in any words, as many messages as you want, for free. That window re-opens every single time they send another message. A conversation that keeps going never leaves it. **Templates are not part of the chatbot at all** — they are only the cold tap on the shoulder in section 06, which is a separate, optional feature.

So the design goal is simple: **get the customer to send the first message**, and after that nothing stands between you and a completely natural conversation.

02 · Let the customer open the thread

You do not need to cold-message anyone to start a WhatsApp conversation. You need them to tap a link. WhatsApp has a standard "click to chat" URL that opens the app with your number and a message already typed:

```
https://wa.me/919825041200?text=Hi%20AURUM%2C%20about%20my%20enquiry%20AUR-7Q2K4M
```

They tap it, they see the pre-filled text, they hit send. That single tap makes **them** the initiator — the 24-hour window opens, no template was involved, and your AI can now talk like a human being.

Where the pre-filled reference earns its keep

Notice the enquiry ref in the URL. When that message lands on your webhook it carries `AUR-7Q2K4M`, which resolves straight to the customer, their car, their chosen service and everything they typed — without asking a single question. That is the bridge from form to WhatsApp, and it costs nothing.

Concretely, on this site

After the form submits, alongside the chat that opens today, add a **"Continue on WhatsApp"** button carrying that deep link. The customer chooses their channel; either way the assistant already knows them. A QR of the same link works in the studio, on an invoice, or in an Instagram bio — and every one of those routes is template-free, because in every one of them the customer tapped first.

03 · Making it feel human

This is the actual work. The platform will happily let you send natural text — what makes people feel they are talking to a person is **how** you send it. Four things do almost all of it.

1. Send several short messages, not one paragraph

Nothing marks a bot faster than a tidy five-line reply arriving as a single block. People type in fragments and hit send early. So take the model's answer and split it on sentence or paragraph boundaries into two or three messages, sent in sequence with a pause between.

Instead of one message:

```
"Coating alone won't help there. Those swirls need correction first, otherwise you're sealing them in for five years. For a Creta, correction runs ₹12,000–55,000 and coating ₹25,000–95,000. The exact figure comes after we gauge the paint."
```

Send three:

```
→ "Coating alone won't fix that one, unfortunately." [pause 1.4s]
→ "Those swirls need correcting first – otherwise you're sealing them in under the coating for five years." [pause 2.1s]
→ "For a Creta you're looking at ₹12,000–55,000 for the correction and ₹25,000–95,000 for the coating. Exact number comes after we see the paint."
```

2. Type at a human speed

Delay each message by roughly how long it would take to type — around 45–60 words per minute, so a 20-word message waits about 1.5–2 seconds. Cap it at 4–5 seconds so nobody is left staring. An instant reply to a long question is the single loudest bot signal there is, and it costs you nothing to wait.

Show the typing indicator while they wait. The Cloud API lets you mark the incoming message as read and start a typing indicator in the same call — it runs for up to about 25 seconds or until your message lands. Mark as read first, too: a bot that replies without the double blue tick appearing looks wrong in a way people notice without being able to say why.

3. Write the tone into the system prompt

`knowledge.md` already forbids markdown, exclamation marks and six-paragraph answers. WhatsApp needs a little more on top:

- **No markdown at all.** WhatsApp does not render `**bold**` the way the web does — asterisks show up literally and instantly look broken.
- **No bullet lists.** Nobody types a bulleted list into WhatsApp.
- **Contractions.** "won't", "you're", "that's". Their absence reads formal and synthetic.
- **Answer, then ask one thing.** Real conversations volley. Dumping the complete answer and stopping is a brochure.
- **Short acknowledgements are allowed.** "Got it." on its own is very human and no bot ever does it.
- **Never open with "Thank you for contacting AURUM Detail Studio."** That sentence is a corporate autoresponder and everyone knows it.

4. Respect the clock

A studio that answers instantly and flawlessly at 3:40am is obviously not a person. Two honest options: queue non-urgent replies until opening time, or let the assistant answer but acknowledge the hour — "we're shut right now, but I can give you the bands and someone will call in the morning." The second keeps the usefulness and drops the uncanniness.

What not to bother with

Deliberate typos, "umm", or fake corrections. They read as gimmicks the moment anyone notices the pattern, and they undercut a studio whose whole pitch is precision. Brevity, pacing and volleying do the work; theatre does not.

04 · The bot itself

The thinking is already built. `buildSystemPrompt()` assembles `knowledge.md` plus the customer's enquiry, and that is exactly what the WhatsApp bot should send to DeepSeek. WhatsApp does not need a second brain — it needs a second **transport**.

1 Meta POSTs your webhook

The sender arrives as `919825041200`.

2 Verify, then answer 200 immediately

Check `X-Hub-Signature-256`, return 200 in milliseconds, do the AI call afterwards. Meta retries anything slow, and a retry means the customer gets two replies.

3 Resolve the customer

Strip the `91` to the bare 10 digits the database already stores. Found → load their enquiry and conversation. Not found → greet and ask which car; they are still a lead.

4 Mark read + start typing

One API call, before you think. This is the pacing from section 03.

5 Call DeepSeek without streaming

WhatsApp has no partial messages, so drop `stream: true` and take the finished reply. The typewriter has no equivalent here — the typing indicator plays that role.

6 Chunk, pace, send

Split into two or three messages, delay each by its typing time, send in order.

7 Persist to the same tables

Write to the existing `Message` table tagged `channel: WHATSAPP`. One transcript per customer across both channels — the studio sees one conversation, not two.

Why this is the good version

A customer can start on the website, drop off, tap through to WhatsApp days later, and the assistant picks up mid-thread — because both channels read the same `Conversation` rows. It only works because the phone number was normalised at the door, which the form already does.

05 · "Are you a bot?"

Someone will ask, usually in the first few messages. Decide the answer now, because the model will improvise one otherwise.

The recommendation: sound human, but do not lie about being one. Being caught in the lie is far more damaging than the admission — and a customer about to spend ₹96,000 on their car is exactly the customer who will test it. The good answer is honest and immediately useful, so the question dies rather than becoming the conversation:

```
"I'm the studio's assistant – AI, but I've got your enquiry  
in front of me and I know the pricing properly. Anything I  
can't settle, I'll get one of the team to call you."
```

That keeps the trust, keeps the momentum, and sets up the handoff. Put it in `knowledge.md` as a fixed rule rather than leaving it to chance.

Handoff

Escalate to a human on: an angry customer, a complaint about completed work, anything about a firm date or an exact figure, and any explicit "can I talk to a person". The existing knowledge base already says escalate rather than argue; on WhatsApp the handoff is easier, because a real person can simply pick up the same thread.

06 · Cold outbound — the only place templates appear

This is the optional half: reaching a lead who filled the form and then went quiet. Here, and only here, the template rules apply — because you are speaking first into a thread the customer never opened.

Cloud API or a BSP

Path	Good	Bad
Meta Cloud API direct	No middleman fee — you pay Meta's per-message rate only. Full control. Hosted by Meta, so nothing to run.	You build everything: templates, webhook, retries, opt-out, dashboards. Business verification is on you.
BSP AiSensy, Interakt, Wati, Gupshup, Twilio	Live in days. Template manager, shared team inbox, broadcast UI, analytics. They walk you through verification.	Monthly fee on top of Meta's per-message cost. Their API, their limits. Bot logic often has to live in their flow builder unless they expose webhooks.

Recommendation

Cloud API direct. The deciding factor is Feature B: you want the bot answering from `knowledge.md` with the customer's enquiry in the prompt — the exact system already built and tested. A BSP's visual flow builder actively fights that; you would end up using them as a dumb pipe while still paying the monthly fee. The one reason to pick a BSP is if the studio needs a **human team inbox** to take over conversations, which is genuinely painful to build yourself.

07 · Prerequisites, in order

1

Meta Business Account

business.facebook.com. Complete

Business Verification

— documents proving the business exists. Start here; it is the slowest step and everything else waits on it.

2

WhatsApp Business Account (WABA)

Created inside Meta Business Suite and linked to a Meta app in developers.facebook.com.

3

A dedicated phone number

It

cannot

be active on the normal WhatsApp or WhatsApp Business app. If the studio's number is already on WhatsApp you must delete that account first, or buy a new number. Decide this early — it is the most common blocker.

4

Display name approval

"AURUM Detail Studio" must match the verified business name or it gets rejected.

5

Permanent access token

Create a System User in Business Settings and issue a token with `whatsapp_business_messaging` and `whatsapp_business_management`. The token from the dev console expires in 24 hours — do not build on it.

6

A public HTTPS webhook

Meta must reach your server. Localhost will not do — use the deployed URL, or `ngrok` while developing.

You end up with four secrets:

```
WHATSAPP_TOKEN=          # permanent system-user token
WHATSAPP_PHONE_NUMBER_ID= # the sending number's id, not the number
WHATSAPP_WABA_ID=
WHATSAPP_VERIFY_TOKEN=   # any random string you invent; Meta echoes it back
WHATSAPP_APP_SECRET=     # to verify webhook signatures
```

07 · Prerequisites, in order

1 Meta Business Account

business.facebook.com. Complete

Business Verification

— documents proving the business exists. Start here; it is the slowest step and everything else waits on it.

2 WhatsApp Business Account (WABA)

Created inside Meta Business Suite and linked to a Meta app in developers.facebook.com.

3 A dedicated phone number

It

cannot

be active on the normal WhatsApp or WhatsApp Business app. If the studio's number is already on WhatsApp you must delete that account first, or buy a new number. Decide this early — it is the most common blocker.

4 Display name approval

"AURUM Detail Studio" must match the verified business name or it gets rejected.

5 Permanent access token

Create a System User in Business Settings and issue a token with `whatsapp_business_messaging` and `whatsapp_business_management`. The token from the dev console expires in 24 hours — do not build on it.

6 A public HTTPS webhook

Meta must reach your server. Localhost will not do — use the deployed URL, or `ngrok` while developing.

You end up with four secrets:

```
WHATSAPP_TOKEN=          # permanent system-user token
WHATSAPP_PHONE_NUMBER_ID= # the sending number's id, not the number
WHATSAPP_WABA_ID=
WHATSAPP_VERIFY_TOKEN=   # any random string you invent; Meta echoes it back
WHATSAPP_APP_SECRET=     # to verify webhook signatures
```

The nudge, concretely

Goal: someone submits the form, never sends a message in the site chat, and a while later gets a WhatsApp from the studio.

Defining "never talked to the chatbot"

Precisely: the customer has an `Enquiry`, and their `Conversation` contains **zero messages with role USER**. The assistant's own opening does not count — the test is whether the human said anything back. That is a single query against the existing schema.

Why it needs a scheduler

You cannot decide at submit time; you have to wait and see whether they engage. So the flow is: on submit, write a **scheduled outbox row** due in ~20–30 minutes. A worker wakes up, re-checks the condition, and sends only if it still holds. Options: Vercel Cron hitting a protected route every 5 minutes, a Postgres-backed queue polled by a worker, or a proper queue (BullMQ + Redis). For this volume a cron route reading an outbox table is the honest choice.

The template

Submitted once, approved, then filled with variables per send:

```
Hi {{1}}, this is AURUM Detail Studio in Ahmedabad.

We have your enquiry {{2}} about {{3}} for your {{4}}.

Reply here and I can give you the price band and what
the inspection involves. If now is not a good time,
reply STOP and we will not message again.
```

UTILITY — it references an enquiry they submitted, states a fact, and offers to continue. No offer, no urgency, no adjectives. That is what gets a Utility approval rather than a Marketing rejection.

Sending it

```
POST https://graph.facebook.com/v21.0/{PHONE_NUMBER_ID}/messages
Authorization: Bearer {WHATSAPP_TOKEN}

{
  "messaging_product": "whatsapp",
  "to": "919825041200",
  "type": "template",
  "template": {
    "name": "enquiry_follow_up",
    "language": { "code": "en" },
    "components": [{
      "type": "body",
      "parameters": [
        { "type": "text", "text": "Rohan" },
        { "type": "text", "text": "AUR-7Q2K4M" },
        { "type": "text", "text": "ceramic coating" },
        { "type": "text", "text": "2022 Hyundai Creta" }
      ]
    }]
  }
}
```



```
}  
}
```

Make it idempotent

Store the returned message id and mark the outbox row sent **in the same transaction** as the send. A cron that retries a half-finished row will message the same customer twice, which is precisely the behaviour that destroys a number's quality rating.

08 · Schema changes

```
model Customer {  
  // ... existing fields  
  whatsappOptIn    Boolean    @default(false)  
  whatsappOptInAt   DateTime?  
  whatsappOptInFrom String?    // page the consent was given on  
  whatsappBlocked   Boolean    @default(false) // they replied STOP  
}  
  
model Message {  
  // ... existing fields  
  channel    MessageChannel @default(WEB)  
  externalId String?         @unique // Meta's message id – dedupes retries  
}  
  
enum MessageChannel { WEB WHATSAPP }  
  
/// Scheduled outbound WhatsApp. A row is a promise to send later,  
/// re-checked at send time in case they engaged in the meantime.  
model WhatsappOutbox {  
  id          String    @id @default(cuid())  
  customerId   String  
  enquiryId    String  
  templateName String  
  payload      Json  
  dueAt        DateTime  
  sentAt       DateTime?  
  failedAt     DateTime?  
  attempts     Int      @default(0)  
  lastError    String?  
  externalId   String?  
  
  @@index([dueAt, sentAt])  
}
```

Files this adds

File	Does
<code>src/lib/whatsapp.ts</code>	Cloud API client: <code>sendTemplate()</code> , <code>sendText()</code> , signature verification.
<code>src/app/api/whatsapp/route.ts</code>	GET verifies the webhook, POST receives messages.

<code>src/app/api/cron/whatsapp/route.ts</code>	Drains the outbox. Protected by a secret header.
<code>src/lib/deepseek.ts</code>	Add a non-streaming <code>complete()</code> beside the streaming path.
<code>enquiry/route.ts</code>	Write the outbox row when consent was given.

09 · What it costs

Item	Roughly
Cloud API access	Free
The whole chatbot — every reply inside the 24h window	Free
Utility template (India)	Fractions of a rupee to ~₹0.2–0.4 per message
Marketing template (India)	Several times the utility rate — roughly ₹0.7–1.0
BSP platform fee, if used	~₹1,000–3,000 / month
DeepSeek per reply	Negligible — well under ₹0.5 per conversation

Meta has moved toward per-message pricing and periodically revises rates and free allowances by country. Treat the above as orders of magnitude and check the current India rate card before budgeting. At nine cars a week the total is small either way; the platform fee, not the messages, is the real cost.

10 · Gotchas and build order

Nine things that will bite you

1. **The 24-hour window.** Half of all "why won't it send" is this — and it only ever applies to messages you start.
2. **The number is already on WhatsApp.** Discover this before you plan a launch date, not during one.
3. **Template rejection.** Usually wrong category, or variables at the very start/end of the body. Keep a plain sentence around each `{{n}}`.
4. **Duplicate webhooks.** Meta delivers at-least-once. Dedupe on the message id — that is what `Message.externalId @unique` is for.
5. **Slow webhook responses.** Return 200 in milliseconds and do the AI call after, or Meta retries and your customer gets two replies.
6. **Quality rating.** Messaging limits start low and tier up on good behaviour. Blocks push you the other way, fast.
7. **No streaming.** The typewriter has no WhatsApp equivalent — the typing indicator plus chunked sends is the substitute.
8. **Opt-out is not optional.** Detect STOP and stop, permanently.

9. **Business verification takes real time.** Days, occasionally longer. It is the long pole — start it first.

Build order

- 1 Paperwork, in parallel with everything**
Business verification, WABA, sort out the phone number. Blocks all sends; blocks no code.
- 2 Consent**
Checkbox on the form, four columns on `Customer` . Ship it early so real consent accumulates while the rest is built.
- 3 The chatbot first, the nudge second**
The inbound bot needs no templates and no approvals — you can test it by messaging your own number the day the WABA exists. It is also the part you actually want. Get it working end to end, including chunking and pacing, before touching outbound.
- 4 Submit the template**
Do it as soon as the WABA exists — approval is a waiting game you can overlap with development.
- 5 Cold outbound last, if at all**
Outbox table, cron route, template. Worth asking whether you need it: a "Continue on WhatsApp" button after the form may capture most of the same people with none of the approval overhead. Test on the studio's own numbers before it touches a real lead.

Realistically: a few days of engineering spread across one to two weeks of waiting on Meta. The waiting dominates, which is the argument for starting the verification today and writing the code around it.

AURUM Detail Studio is a fictional business built to showcase a marketing site. This document describes work that has **not** been implemented.